Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Extadas e Informática
Algoritmos e Estruturas de Dados II

Trabalho Prático I

1 Regras Básicas

1. Estude bastante cada par de entrada/saída.
2. Todos os programas serão desenvolvidos nas linguagens Java ou C.
3. Todas as apresentações de trabalho devem ser feitas no Linux usando o editor VIM.
4. Como o combinado não sai caro: As cópias de trabalho serão encaminhadas para o colegiado. É considerado cópia a inclusão de código não autoral (não produzido pelo próprio alunol), incluindo uso de IA. O aluno perderá 1 ponto para cada método não comentado ou com comentário inútil.
5. As exceções devem ser perguntadas/discutidas/negociadas com o professor.
6. Neste trabalho, nas questões em Java, só podem ser utilizados os tipos de dados primitivos e as classes MyIO, String, Scanner, sendo que, na classe String, os únicos métodos permitidos são *char charAt(int)* e *int length()*. Nas questões em C, só podem ser utilizados os tipos de dados primitivos e os métodos disponíveis em *stdio.h* e *stdlib.h* .
7. Para cada questão, deve ser submetido apenas um arquivo (.java ou .c). Essa regra será necessária para a submissão de trabalhos no Verde e no identificador de plágio utilizado na disciplina.
8. Use o padrão Java para comentários.
9. Na leitura de um número inteiro, se seu valor for vazio ou não número, ele recebe zero.
10. O arquivo **readme.txt** contém dicas sobre Linux, VIM e redirecionamento de entrada/saída.
11. Para contar as letras, consoantes e vogais desconsidere acentos e cedilha, ou seja, consideramos somente os caracteres cujos códigos ASCII estiverem entre ‘A’ e ‘Z’ ou ‘a’ e ‘z’.

12. Para cada exercício: faça (entende-se análise, implementação e comentários), teste (várias vezes) e submeta no Verde. Os exercícios não testados/comentados serão penalizados.

13. A correção será automática através do Verde e de entrevista com o aluno nas aulas de laboratório.

14. A correção sempre será feita na versão do Linux disponível nos laboratórios. Qualquer problema devido ao uso de outras arquiteturas será de responsabilidade do aluno.

15. Fique atento ao **charset** dos arquivos de entrada e saída.

2 Dicas sobre Recursividade

1. Um método recursivo pode ser usado como uma estrutura de repetição e um erro básico consiste em confundir tais estruturas com os métodos recursivos. Para evitar esse erro, neste trabalho, não podemos usar estruturas de repetição nos métodos recursivos.

2. Outro erro básico consiste em criar variáveis globais (em Java, atributos) para darem suporte à recursividade. Para evitar esse erro, neste trabalho, não podemos usar variáveis globais.

3. Um terceiro erro básico consiste em passar o valor de retorno como parâmetro. Para evitar esse erro, neste trabalho, não podemos passar parâmetros extras aos do enunciado, exceto um contador para recursividade. O método abaixo contém um exemplo desse erro.

```
boolean contarLetrasMinusculas(String s, int i, int resp){
    int resp;
    if(i == s.length()){
        resp = 0;
    }
    else if ((s.charAt(i) >= 'a' && s.charAt(i) <= 'z') == false){
        resp = contarLetrasMinusculas(s, i + 1, resp + 1);
    }
    else {
        resp = contarLetrasMinusculas(s, i + 1, resp);
    }
    return resp;
}
```

4. Quando criamos um método recursivo, normalmente, passamos um contador como parâmetro. Uma forma elegante de implementar essa solução recursiva consiste em criar dois métodos: o recursivo; e outro que chama o recursivo pela primeira vez e inicializa o valor do contador. Neste trabalho, devemos usar essa técnica em todos os métodos recursivos. Veja o exemplo abaixo:

```

boolean somenteLetrasMinusculas(String s){
    return somenteLetrasMinusculas(s, 0);
}
boolean somenteLetrasMinusculas(String s, int i){
    boolean resp;
    if(i == s.length()){
        resp = true;
    }
    else if ((s.charAt(i) >= 'a' && s.charAt(i) <= 'z') == false){
        resp = false;
    }
    else {
        resp = somenteLetrasMinusculas(s, i + 1);
    }
    return resp;
}
```

5. Uma boa prática de programação é colocar um único *return* em cada método. Por exemplo, o método anterior é mais indicado que a versão abaixo.

```java

boolean somenteLetrasMinusculas(String s, int i){
    if(i == s.length()){
        return true;
    }
    else if ((s.charAt(i) >= 'a' && s.charAt(i) <= 'z') == false){
        return false;
    }
    else {
        return somenteLetrasMinusculas(s, i + 1);
    }
}
```

3 Questões

1. **Ciframento de César** - O Imperador Júlio César foi um dos principais nomes do Império Romano. Entre suas contribuições, temos um algoritmo de criptografia chamado "Ciframento de César". Segundo os historiadores, César utilizava esse algoritmo para criptografar as mensagens que enviava aos seus generais durante as batalhas. A ideia básica é um simples deslocamento de caracteres. Assim, por exemplo, se a chave utilizada para criptografar as mensagens for 3, todas as ocorrências do caractere 'a' são substituídas pelo caractere 'd', as do 'b' por 'e', e assim sucessivamente. Crie um método **iterativo** que recebe uma string como parâmetro e retorna outra contendo a entrada de forma cifrada. Neste exercício, suponha a chave de ciframento três. Na saída padrão, para cada linha de entrada, escreva uma linha com a mensagem criptografada.

2. **Alteração Aleatória** - Crie um método **iterativo** que recebe uma string, sorteia duas letras minúsculas aleatórias (código ASCII $\geq$ 'a' e $\leq$ 'z'), substitui todas as ocorrências da primeira letra na string pela segunda e retorna a string com as alterações efetuadas. Na saída padrão, para cada linha de entrada, execute o método desenvolvido nesta questão e mostre a string retornada como uma linha de saída. Abaixo, observamos um exemplo de entrada supondo que para a primeira linha as letras sorteados foram o 'a' e o 'q'. Para a segunda linha, foram o 'e' e o 'k'.

EXEMPLO DE ENTRADA:	EXEMPLO DE SAÍDA:
o rato roeu a roupa do rei de roma	*o rqto roeu q roupq do rei de romq*
e qwe qwe qwe ewq ewq ewq	*k qwk qwk qwk kwq kwq kwq*
FIM	

A classe Random do Java gera números (ou letras) aleatórios e o exemplo abaixo mostra uma letra minúscula na tela. Em especial, destacamos que: i) *seed* é a semente para geração de números aleatórios; ii) nesta questão, por causa da correção automática, a *seed* será quatro; iii) a disciplina de Estatística e Probabilidade faz uma discussão sobre "aleatório".

```
Random gerador = new Random();
gerador.setSeed(4);
System.out.println((char)('a' + (Math.abs(gerador.nextInt()) % 26)));
```

3. **Is** - Crie um método **iterativo** que recebe uma string e retorna true se a mesma é composta somente por vogais. Crie outro método **iterativo** que recebe uma string e retorna true se a mesma é composta somente por consoantes. Crie um terceiro método **iterativo** que recebe uma string e retorna true se a mesma corresponde a um número inteiro. Crie um quarto método **iterativo** que recebe uma string e retorna true se a mesma corresponde a um número real. Na saída padrão, para cada linha de entrada, escreva outra de saída da seguinte forma X1 X2 X3 X4 onde cada Xi é um booleano indicando se a é entrada é: composta somente por vogais (X1);

composta somente somente por consoantes (X2); um número inteiro (X3); um número real (X4). Se Xi for verdadeiro, seu valor será SIM, caso contrário, NÃO.

4. **Inversão de String** - Crie um método **iterativo** que recebe uma string como parâmetro e retorna a string invertida. Na saída padrão, para cada linha de entrada, escreva uma linha de saída com a string invertida. Por exemplo, se a entrada for "abcde", a saída deve ser "edcba".

5. **Soma de Dígitos** - Crie um método **recursivo** que recebe um número inteiro como parâmetro e retorna a soma de seus dígitos. Na saída padrão, para cada linha de entrada, escreva uma linha de saída com o resultado da soma dos dígitos. Por exemplo, se a entrada for 12345, a saída deve ser 15.

6. **Verificação de Anagrama** - Crie um método **iterativo** que recebe duas strings como parâmetros e retorna true se as strings são anagramas uma da outra, ou false caso contrário. Na saída padrão, para cada par de strings de entrada, escreva uma linha de saída com SIM/NÃO indicando se as strings são anagramas. Por exemplo, se as entradas forem "listen" e "silent", a saída deve ser SIM.

7. **Substring Mais Longa Sem Repetição** - Crie um método **iterativo** que recebe uma string como parâmetro e retorna o comprimento da substring mais longa sem caracteres repetidos. Na saída padrão, para cada linha de entrada, escreva uma linha de saída com o comprimento da substring mais longa sem repetição. Por exemplo, se a entrada for "abcabcbb", a saída deve ser 3 (correspondendo à substring "abc").

8. **Validação de Senha** - Crie um método **iterativo** que recebe uma string como parâmetro e retorna true se a string é uma senha válida, ou false caso contrário. Uma senha é considerada válida se contém pelo menos 8 caracteres, incluindo pelo menos uma letra maiúscula, uma letra minúscula, um número e um caractere especial (por exemplo, !, @, #, etc.). Na saída padrão, para cada linha de entrada, escreva uma linha de saída com SIM/NÃO indicando se a senha é válida. Por exemplo, se a entrada for "Senha123!"s, a saída deve ser SIM.

9. **Ciframento de César (RECURSIVO)** - Refaça a questão **Ciframento de César** de forma **recursiva**.

10. **Is (RECURSIVO)** - Refaça a questão **Is** de forma **recursiva**.

11. **Inversão de String (RECURSIVO)** - Refaça a questão **Inversão de String** de forma **recursiva**.

12. **Soma de Dígitos (RECURSIVO)** - Refaça a questão **Soma de Dígitos** de forma **recursiva**.